SAVIKA

PREMIUM SPICES

Master Platform Document

Product Requirements | Technical Architecture | Design System | Environment Setup

Everything your developer and AI IDE need to build Savika end-to-end.

Version 1.0 | March 2026 | Confidential

Document Section	What It Covers
Part 1 - Product Requirements (PRD)	Goals, user personas, feature specs, admin panel, e-commerce flow, notifications
Part 2 - Technical Architecture (TRD)	Stack decisions, database schema, API routes, auth flow, security, Lighthouse targets
Part 3 - Design System	Fonts, colours, spacing, component library, animation rules, responsive breakpoints
Part 4 - Environment Setup Guide	Service-by-service setup, every env variable, copy-paste code, go-live checklist

Decision	Choice	Reason
Frontend	Next.js (existing codebase)	No rebuild needed
Backend + Auth + DB	Supabase	Free tier, Next.js native, built-in realtime + storage
Payments	Razorpay	India-first, UPI + cards + wallets
Email	Resend	3k free/month, 5-min Next.js setup
WhatsApp + SMS	Twilio	Reliable, sandbox available, India-ready

Decision	Choice	Reason
Browser Push	VAPID + web-push	Free, no third party, works offline
Image Storage	Supabase Storage	Free 1GB, CDN included, same dashboard
Hosting	Vercel free tier	Native Next.js, zero config
Global Font	Poppins	Replacing all existing fonts site-wide

PART 1

Product Requirements Document

What we are building and why

1.1 Product Overview

Savika Premium Spices is an Indian e-commerce platform selling authentic masala blends, exotic and rare spices. The platform targets Indian consumers with a premium shopping experience, full admin control, and user data capture for logistics and marketing.

This document defines all features to be added to the existing Next.js codebase without a full rebuild. The scope covers the user-facing storefront, admin dashboard, authentication system, payment flow, notification system, and CMS-style content management.

1.2 Goals

- Deliver a production-ready e-commerce storefront with full cart, checkout, and order tracking
- Build a fully responsive Super Admin dashboard with content, product, order and customer management
- Implement secure auth for users (Email + Google OAuth) and admin via Supabase Auth
- Enable real-time order push notifications to admin across Browser Push, WhatsApp, and Email
- Auto-capture complete user data on registration for delivery and marketing use
- Enable one-click CSV export of all user and order data from the admin panel
- Achieve Lighthouse score of 90 or above on Performance, SEO, Accessibility, Best Practices
- Apply Poppins font globally across every page and component
- Remove all em dashes from the entire codebase and replace with appropriate alternatives
- Implement lazy loading and skeleton screens for all image-heavy pages

1.3 Out of Scope

- International shipping or multi-currency support
- Native mobile app (iOS or Android)
- Loyalty points or rewards programme
- Multi-vendor or marketplace functionality
- Inventory forecasting or supplier management

1.4 User Personas

Persona A: End User (Shopper)

- Indian consumers aged 22 to 50, primarily urban
- Purchases masala blends, exotic spices for home cooking or gifting
- Expects a fast, mobile-first experience with Google login support

- Wants order tracking and delivery address saved for repeat purchases

Persona B: Super Admin

- Single administrator with full access to all dashboard sections
- Manages products, orders, customers, coupons, and site content
- Receives instant notifications on every new order via 3 channels
- Downloads customer and order data as CSV for logistics and marketing
- Updates homepage banners, section text, icons, and background images without developer involvement

1.5 Feature Specifications

1.5.1 User Authentication

Feature	Details
Sign Up	Email + Password with email verification via Resend. Auto-saves name, email, mobile, address to profiles table on completion.
Log In	Email + Password. Supabase handles sessions with secure httpOnly cookies.
Google OAuth	One-click Google login. Profile auto-created in Supabase on first login.
Forgot Password	Email reset link sent via Supabase Auth built-in flow.
Auth Guards	Protected routes for /account, /orders, /checkout. Redirect to /login if unauthenticated.
Admin Auth	Separate admin login page at /admin/login. Role checked against admins table after login.
Session Management	Supabase SSR session via middleware.ts on all routes.

1.5.2 User-Facing E-Commerce Flow

Page / Feature	Specification
Homepage	Hero banners (admin editable), promotional cards (admin editable text, icon, background image), featured products, categories strip.
Product Listing	Grid layout, filter by category and price, search bar, skeleton loaders, lazy-loaded images.
Product Detail	Image gallery (from Supabase Storage), description, price, Add to Cart button, related products.
Cart	Persistent cart via localStorage + Supabase for logged-in users. Coupon code input. Order summary.
Checkout	Pre-filled shipping address from profile. Razorpay payment modal. Order confirmation on success.
Order Tracking	List of user orders with status badges (Pending, Processing, Shipped, Delivered). Timestamps.
User Account	Edit profile: name, mobile, address. All fields saved to Supabase profiles table.
Auth Pages	Sign Up, Log In, Forgot Password. Poppins font. Matching Savika brand colours.

1.5.3 Admin Dashboard Pages

CRITICAL: All admin pages must be fully responsive. Admin panel accessible at /admin. Only authenticated admins can access any /admin route.

Page	Features Required
Dashboard	Summary cards: Total Orders, Revenue, Customers, Products. Recent orders table. Quick actions. Charts for weekly sales.
Orders	Full orders table with status filter. Update order status (Pending, Processing, Shipped, Delivered, Cancelled). View order detail with items and shipping address. Export orders CSV.
Products	List all products. Add new product: name, description, price, category, stock, image upload to Supabase Storage. Edit product inline. Delete product with confirmation. Toggle product active/inactive.
Categories	Add, edit, delete product categories. Assign category image.
Customers	List all registered users: name, email, mobile, address, join date. Search and filter. Export all customers as CSV with one button click.
Coupons	Create coupon: code, percentage discount or flat discount, expiry date. Toggle active/inactive. Delete coupon.
Analytics	Revenue chart (weekly/monthly). Top selling products. Orders by status pie chart. New customer signups trend.
Updates (Content)	Admin-editable homepage sections: upload background image, change icon, change heading text, change subtext for each promotional card. Changes reflect instantly on storefront.
Settings	Admin profile, change password, notification preferences.

1.5.4 Admin Updates Panel (Content Management)

The Updates page gives the admin full control over the two promotional cards visible on the homepage (as shown in the provided screenshots). No developer involvement is needed for any content change.

Editable Element	Control Type	Storage
Section background image	Image upload button (JPG/PNG/WEBP, max 2MB)	Supabase Storage > site-assets bucket
Section icon	Icon picker (from predefined icon set) OR image upload	Supabase Storage > site-assets bucket
Badge text (e.g. Limited Time, Fan Favourite)	Text input field	Supabase site_content table

Editable Element	Control Type	Storage
Headline text (e.g. 50% OFF)	Text input field with formatting	Supabase site_content table
Subheading text (e.g. On all masala blends)	Text input field	Supabase site_content table
Body text (e.g. Use code SPICE50)	Text input field	Supabase site_content table
Button label (e.g. Buy Now)	Text input field	Supabase site_content table
Button link / target URL	URL input field	Supabase site_content table

1.5.5 Notification System

Every time a user places an order and payment is confirmed by Razorpay webhook, ALL three of the following notifications are sent simultaneously to the admin:

Channel	Tool	Message Content
Browser Web Push	VAPID + web-push library	Order ID, customer name, total amount, timestamp
WhatsApp Message	Twilio WhatsApp API	Order ID, customer name, total amount, items summary
Email	Resend	Full order details: items, amounts, shipping address, customer contact

1.5.6 Performance Requirements

Metric	Target	Method
Lighthouse Performance	>=90	Lazy loading, next/image, skeleton screens, font preloading
Lighthouse SEO	>=90	Meta tags, OG tags, structured data, sitemap.xml, robots.txt
Lighthouse Accessibility	>=90	ARIA labels, colour contrast, keyboard navigation, focus rings
Lighthouse Best Practices	>=90	HTTPS, no console errors, valid HTML, CSP headers
Core Web Vitals LCP	<2.5s	Next/image with priority on above-fold images
Core Web Vitals CLS	<0.1	Reserved space for images and dynamic content

Metric	Target	Method
Core Web Vitals FID	<100ms	Minimal blocking JavaScript, deferred non-critical scripts

PART 2

Technical Requirements Document

Architecture, schema, API routes, and security

2.1 Technology Stack

Layer	Technology	Version / Notes
Frontend Framework	Next.js	App Router (existing codebase). Do NOT migrate to Pages Router.
Language	TypeScript	Strict mode. All new files must be .ts or .tsx.
Styling	Tailwind CSS	Existing. Add Poppins via next/font/google.
Database	Supabase (PostgreSQL)	Free tier. Singapore region.
Authentication	Supabase Auth + @supabase/ssr	SSR-safe. Session in httpOnly cookies via middleware.
File Storage	Supabase Storage	Buckets: product-images (public), site-assets (public).
Realtime	Supabase Realtime	Used for live order status updates in admin dashboard.
Payments	Razorpay	npm install razorpay. Checkout.js loaded via next/script lazyOnload.
Email	Resend	npm install resend. Server-side only.
WhatsApp + SMS	Twilio	npm install twilio. Server-side only.
Browser Push	web-push	npm install web-push. VAPID keys self-generated.
Hosting	Vercel	Free hobby tier. Auto-deploys on main branch push.
Image Optimisation	next/image	All product and banner images must use next/image component.
Font	next/font/google (Poppins)	Replace ALL font references site-wide. Subsets: latin, latin-ext.

2.2 Folder Structure

INFO: AI IDE Instruction: The following folder structure must be followed exactly. Create missing folders and files as described.

```
/savika-spices                    <- project root
  /app                            <- Next.js App Router
    /admin                        <- all admin routes (protected)
      /dashboard/page.tsx
      /orders/page.tsx
      /products/page.tsx
      /categories/page.tsx
      /customers/page.tsx
      /coupons/page.tsx
      /analytics/page.tsx
      /updates/page.tsx
      /settings/page.tsx
      /login/page.tsx             <- admin login (separate from user login)
      layout.tsx                  <- admin layout with sidebar
    /auth
      /login/page.tsx
      /signup/page.tsx
      /forgot-password/page.tsx
      /callback/route.ts          <- OAuth callback handler
    /account/page.tsx             <- user profile page
    /orders/page.tsx              <- user order history
    /products/page.tsx            <- product listing
    /products/[id]/page.tsx       <- product detail
    /cart/page.tsx
    /checkout/page.tsx
    /api
      /orders/create/route.ts
      /orders/webhook/route.ts    <- Razorpay webhook
      /products/route.ts
      /push/subscribe/route.ts
      /push/send/route.ts
      /notify/order/route.ts
      /export/customers/route.ts
      /export/orders/route.ts
      /content/route.ts           <- site_content CRUD
    layout.tsx                    <- root layout with Poppins font
  /components
    /ui                           <- shared UI: Button, Input, Badge, Modal, Skeleton
    /admin                        <- admin-only components
    /storefront                   <- user-facing components
  /lib
    /supabase
      client.ts                   <- browser client
      server.ts                   <- server client
      admin.ts                    <- service role client (server only)
    /razorpay.ts
    /resend.ts
    /twilio.ts
    /webpush.ts
  /types
    index.ts                      <- all TypeScript interfaces
  /public
    /sw.js                        <- service worker for web push
    /icons                        <- app icons for PWA
```

```
middleware.ts                    <- Supabase session refresh on every request
.env.local                       <- all secrets (never commit)
.gitignore                       <- must include .env.local
```

2.3 Database Schema

CRITICAL: AI IDE Instruction: Run this entire SQL block in Supabase SQL Editor before writing any application code. This creates all required tables, RLS policies, and triggers.

```
-- ────────────────────────────────────────────────────────────────────
-- SAVIKA PREMIUM SPICES - Complete Database Schema
-- Run once in Supabase SQL Editor
-- ────────────────────────────────────────────────────────────────────

-- PROFILES (extends auth.users)
create table public.profiles (
  id           uuid references auth.users(id) on delete cascade primary key,
  full_name    text,
  email        text unique,
  mobile       text,
  address      jsonb,
  avatar_url   text,
  created_at   timestamptz default now(),
  updated_at   timestamptz default now()
);

-- CATEGORIES
create table public.categories (
  id           uuid default gen_random_uuid() primary key,
  name         text not null,
  slug         text unique not null,
  image_url    text,
  created_at   timestamptz default now()
);

-- PRODUCTS
create table public.products (
  id           uuid default gen_random_uuid() primary key,
  name         text not null,
  slug         text unique not null,
  description  text,
  price        numeric(10,2) not null,
  compare_price numeric(10,2),
  stock        integer default 0,
  category_id  uuid references public.categories(id),
  image_urls   text[] default '{}',
  is_active    boolean default true,
  tags         text[] default '{}',
  created_at   timestamptz default now()
);

-- ORDERS
create table public.orders (
  id                   uuid default gen_random_uuid() primary key,
  user_id              uuid references public.profiles(id),
  items                jsonb not null,
  subtotal             numeric(10,2) not null,
  discount             numeric(10,2) default 0,
  total                numeric(10,2) not null,
  coupon_code          text,
```

```
  status                text default 'pending',
  razorpay_order_id     text,
  razorpay_payment_id   text,
  shipping_address      jsonb,
  created_at            timestamptz default now(),
  updated_at            timestamptz default now()
);

-- COUPONS
create table public.coupons (
  id                uuid default gen_random_uuid() primary key,
  code              text unique not null,
  discount_percent  integer,
  discount_flat     numeric(10,2),
  min_order_value   numeric(10,2) default 0,
  is_active         boolean default true,
  expires_at        timestamptz,
  created_at        timestamptz default now()
);

-- SITE CONTENT (admin-editable content)
create table public.site_content (
  key          text primary key,
  value        jsonb not null,
  updated_at   timestamptz default now()
);

-- ADMINS
create table public.admins (
  id           uuid references auth.users(id) on delete cascade primary key,
  email        text unique not null,
  full_name    text,
  role         text default 'admin',
  created_at   timestamptz default now()
);

-- PUSH SUBSCRIPTIONS
create table public.push_subscriptions (
  id           uuid default gen_random_uuid() primary key,
  subscription jsonb not null,
  created_at   timestamptz default now()
);

-- ── ROW LEVEL SECURITY ──────────────────────────────────────────────
alter table public.profiles enable row level security;
alter table public.orders enable row level security;
alter table public.products enable row level security;
alter table public.coupons enable row level security;

create policy "Own profile" on public.profiles
  for all using (auth.uid() = id);

create policy "Own orders" on public.orders
  for all using (auth.uid() = user_id);

create policy "Public products" on public.products
  for select using (true);
```

```
create policy "Active coupons read" on public.coupons
  for select using (is_active = true);

-- ── AUTO-SAVE USER PROFILE ON SIGNUP ──────────────────────────
create or replace function public.handle_new_user()
returns trigger as $$
begin
  insert into public.profiles (id, email, full_name)
  values (
    new.id,
    new.email,
    coalesce(new.raw_user_meta_data->>'full_name', new.email)
  ) on conflict (id) do nothing;
  return new;
end;
$$ language plpgsql security definer;

create or replace trigger on_auth_user_created
  after insert on auth.users
  for each row execute procedure public.handle_new_user();

-- ── SEED SITE CONTENT DEFAULTS ────────────────────────────────
insert into public.site_content (key, value) values
('promo_card_1', '{
  "badge": "Limited Time",
  "headline": "50% OFF",
  "subheading": "On all masala blends",
  "body": "Use code SPICE50 at checkout",
  "button_label": "Buy Now",
  "button_url": "/products",
  "bg_image": null,
  "icon": "jar"
}'),
('promo_card_2', '{
  "badge": "Fan Favourite",
  "headline": "50% OFF",
  "subheading": "Exotic & Rare spices",
  "body": "Desi. Vegan. Powerful.",
  "button_label": "Buy Now",
  "button_url": "/products?category=exotic",
  "bg_image": null,
  "icon": "diamond"
}');
```

2.4 API Routes Reference

CRITICAL: AI IDE Instruction: Create all routes in /app/api/. Every route must validate authentication before processing. Service-role Supabase client must NEVER be imported in client components.

Route	Method	Auth Required	Purpose
api/orders/create	POST	User	Create Razorpay order, save to DB, trigger notifications
api/orders/webhook	POST	Razorpay signature	Confirm payment, update order status, fire all notifications
api/orders/[id]	GET / PATCH	Admin	Get order detail or update status
api/products	GET / POST	GET: Public, POST: Admin	List or create products
api/products/[id]	GET / PATCH / DELETE	GET: Public, rest: Admin	CRUD single product
api/content	GET / PATCH	GET: Public, PATCH: Admin	Read or update site_content table
api/push/subscribe	POST	Admin	Save browser push subscription to DB
api/push/send	POST	Internal	Send web push to all subscribed browsers
api/notify/order	POST	Internal	Fire Email + WhatsApp + Web Push simultaneously
api/export/customers	GET	Admin	Stream CSV of all profiles
api/export/orders	GET	Admin	Stream CSV of all orders
api/upload/image	POST	Admin	Upload image to Supabase Storage, return public URL
auth/callback	GET	None	Handle Supabase OAuth redirect, exchange code for session

2.5 Authentication Flow

2.5.1 User Auth Flow

- User visits /auth/login or /auth/signup
- Email/password: Supabase Auth creates session, middleware.ts refreshes on every request

- Google OAuth: redirects to Google, returns to /auth/callback, Supabase exchanges code for session
- On first Google login: handle_new_user trigger auto-creates profile row
- Protected pages (/account, /orders, /checkout) check session in server component via createClient()
- Unauthenticated access redirects to /auth/login with return URL preserved

2.5.2 Admin Auth Flow

- Admin visits /admin/login (separate page, separate form)
- Login uses same Supabase Auth email/password
- After login, server checks if user ID exists in admins table
- If not found in admins table: log out immediately and show 'Access Denied' error
- If found: set admin session cookie and redirect to /admin/dashboard
- All /admin/* routes protected by layout.tsx that checks admin session server-side

2.6 Payment Flow

1. User clicks 'Place Order' on checkout page
2. Frontend calls POST /api/orders/create with cart items and shipping address
3. Server creates a Razorpay order via Razorpay SDK and saves order to Supabase with status=pending
4. Server returns Razorpay order_id to frontend
5. Frontend opens Razorpay checkout modal with order_id and NEXT_PUBLIC_RAZORPAY_KEY_ID
6. User completes payment inside Razorpay modal
7. Razorpay sends POST to /api/orders/webhook with payment confirmation
8. Webhook handler verifies signature using RAZORPAY_WEBHOOK_SECRET
9. On valid signature: update order status to paid in Supabase
10. Call POST /api/notify/order which fires Email, WhatsApp, and Browser Push simultaneously
11. Frontend receives success callback from Razorpay modal and redirects to /orders with confirmation

2.7 Security Rules

Rule	Implementation
Service role key never on frontend	SUPABASE_SERVICE_ROLE_KEY used only in /api routes and /lib/supabase/admin.ts

Rule	Implementation
Razorpay secret never on frontend	RAZORPAY_KEY_SECRET used only server-side. Only KEY_ID is NEXT_PUBLIC_
Webhook signature verification	Every Razorpay webhook validated with crypto.createHmac before processing
Admin route protection	All /admin/* pages check admin session in layout.tsx server component
Row Level Security	Supabase RLS enabled on all user-data tables
CSRF protection	Next.js built-in + Supabase SSR cookie handling
Image upload validation	File type (jpg/png/webp only), max 2MB enforced server-side before upload
No em dashes in codebase	Global search and replace all — characters before deployment
Environment variables	All secrets in .env.local locally, Vercel dashboard for production

PART 3

Design System

Fonts, colours, components, animations, and responsive rules

3.1 Typography

CRITICAL: AI IDE Instruction: Remove ALL existing font imports site-wide. Replace with the Poppins configuration below in the root layout.tsx. This applies globally to every page and component.

```
// app/layout.tsx - REPLACE existing font setup with this exactly
import { Poppins } from 'next/font/google'

const poppins = Poppins({
  subsets: ['latin', 'latin-ext'],
  weight: ['300', '400', '500', '600', '700', '800'],
  variable: '--font-poppins',
  display: 'swap',
  preload: true,
})

export default function RootLayout({ children }) {
  return (
    <html lang='en' className={poppins.variable}>
      <body className='font-[--font-poppins]'>{children}</body>
    </html>
  )
}
```

Role	Poppins Weight	Size (Tailwind)	Usage
Display / Hero	800 ExtraBold	text-5xl to text-7xl	Main homepage headlines
Heading H1	700 Bold	text-4xl	Page titles
Heading H2	600 SemiBold	text-2xl to text-3xl	Section headings
Heading H3	600 SemiBold	text-xl	Card titles, sidebar items
Body Regular	400 Regular	text-base (16px)	Descriptions, paragraph text
Body Small	400 Regular	text-sm (14px)	Labels, captions, helper text
Button / Badge	600 SemiBold	text-sm to text-base	All buttons and status badges
Code / Monospace	Courier New fallback	text-sm	Only for code blocks in admin

3.2 Colour Palette

Token	Hex	Tailwind Class	Usage
Primary Gold	#C17F24	text-amber-700 / bg-amber-700	Brand colour, buttons, highlights, badges
Primary Dark Gold	#8B5E16	text-amber-800 / bg-amber-800	Hover states, dark card backgrounds
Light Gold	#F59E0B	text-amber-500	Secondary card (fan favourite) background
Background	#F5F0E8	bg-stone-100	Page background, warm off-white
Surface	#FFFFFF	bg-white	Cards, modals, input backgrounds
Dark	#111111	text-gray-950	Primary text colour
Body Text	#444444	text-gray-700	Paragraph and description text
Muted	#888888	text-gray-500	Placeholders, timestamps, hints
Border	#DDDDDD	border-gray-300	Input borders, card borders, dividers
Success	#1A6B3C	text-green-800	Order confirmed, in stock, success states
Error	#B91C1C	text-red-700	Error messages, out of stock, failed payments
Warning	#B45309	text-amber-700	Low stock warning, pending status
Admin Dark BG	#111111	bg-gray-950	Admin sidebar and header background

3.3 Spacing and Layout

Element	Specification
Page max-width	max-w-7xl (1280px) centred with mx-auto px-4 sm:px-6 lg:px-8
Section vertical spacing	py-12 on mobile, py-20 on desktop
Card padding	p-4 on mobile, p-6 on desktop
Grid columns	1 column mobile, 2 columns tablet (sm), 3-4 columns desktop (lg)
Admin sidebar width	w-64 (256px) fixed on desktop, slide-out drawer on mobile

Element	Specification
Admin content area	Full width minus sidebar. ml-64 on desktop, ml-0 on mobile
Border radius	rounded-2xl for cards and modals. rounded-lg for buttons and inputs. rounded-full for badges
Shadow	shadow-sm for cards at rest. shadow-md on hover. shadow-xl for modals
Input height	h-12 (48px) for all form inputs for touch-friendliness on mobile
Button height	h-12 (48px) for primary buttons. h-10 (40px) for secondary and ghost buttons

3.4 Responsive Breakpoints

Breakpoint	Tailwind Prefix	Target Device	Admin Sidebar
0 - 639px	(default)	Mobile phones	Hidden. Hamburger menu opens drawer overlay.
640px - 767px	sm:	Large phones, small tablets	Hidden. Hamburger menu.
768px - 1023px	md:	Tablets (iPad)	Icon-only collapsed sidebar (w-16).
1024px - 1279px	lg:	Laptops	Full sidebar visible (w-64).
1280px+	xl:	Desktop monitors	Full sidebar + expanded content area.

3.5 Component Specifications

3.5.1 Promotional Cards (Homepage)

The two promotional cards shown in the provided screenshots must follow these exact specifications:

- Container: rounded-3xl, overflow-hidden, relative, min-height 220px on mobile / 280px on desktop
- Background: admin-uploaded image via CSS background-image. Fallback: solid gold gradient if no image.
- Badge pill: absolute top-left, rounded-full, semi-transparent dark background, icon + text
- Headline: Poppins 700, text-5xl, white, with percentage styled larger than OFF text
- Icon circle: absolute right-side, semi-transparent white circle, 80px diameter, icon centred

- Button: white background, gold text, rounded-full, py-3 px-6, Poppins 600, lightning bolt icon left
- Hover effect: scale-105 transform on the entire card, transition-transform duration-300

3.5.2 Skeleton Screens

TIP: Use skeleton screens instead of spinners for all content that loads asynchronously. Skeleton must match the exact shape and size of the content it replaces.

- Product card skeleton: grey rounded rectangle for image, two grey lines for title and price
- Banner skeleton: full-width grey rectangle matching banner height
- Order list skeleton: grey lines matching the table row structure
- Use Tailwind animate-pulse class on all skeleton elements

3.5.3 Animation Rules

Animation	Rule	Tailwind / CSS
Page transitions	Fade in from below on mount	animate-in slide-in-from-bottom-4 duration-300
Card hover	Subtle scale up	hover:scale-105 transition-transform duration-300
Button press	Slight scale down on active	active:scale-95 transition-transform duration-150
Modal open	Fade in + scale up from 95% to 100%	animate-in zoom-in-95 fade-in duration-200
Sidebar items	Fade in sequentially on load	Use staggered animation-delay in CSS
Toast notifications	Slide in from top-right	animate-in slide-in-from-top-2 duration-300
Image load	Fade in after load completes	opacity-0 to opacity-100 on onLoad event
Skeleton to content	Cross-fade when data arrives	Conditional rendering with transition-opacity

PART 4

Environment Setup Guide

Plug in your keys. Everything else runs itself.

INFO: This section is written for AI IDEs like Cursor and Windsurf. Every instruction is atomic and machine-readable. Follow services in order. Do not skip any step.

4.1 Master .env.local Template

Create this file in your Next.js project root immediately. Confirm .env.local is in .gitignore before adding any values. Never commit this file to Git under any circumstances.

```
# ─── SAVIKA PREMIUM SPICES ── .env.local ──────────────────────
# Replace every <PLACEHOLDER> with values from each section below

# ── SUPABASE ──────────────────────────────────────────────────
NEXT_PUBLIC_SUPABASE_URL=<supabase-project-url>
NEXT_PUBLIC_SUPABASE_ANON_KEY=<supabase-anon-key>
SUPABASE_SERVICE_ROLE_KEY=<supabase-service-role-key>

# ── GOOGLE OAUTH ──────────────────────────────────────────────
GOOGLE_CLIENT_ID=<google-oauth-client-id>
GOOGLE_CLIENT_SECRET=<google-oauth-client-secret>

# ── RAZORPAY ──────────────────────────────────────────────────
NEXT_PUBLIC_RAZORPAY_KEY_ID=<razorpay-key-id>
RAZORPAY_KEY_SECRET=<razorpay-key-secret>
RAZORPAY_WEBHOOK_SECRET=<razorpay-webhook-secret>

# ── RESEND (EMAIL) ────────────────────────────────────────────
RESEND_API_KEY=<resend-api-key>
RESEND_FROM_EMAIL=orders@savikaspice.in

# ── TWILIO (WHATSAPP + SMS) ───────────────────────────────────
TWILIO_ACCOUNT_SID=<twilio-account-sid>
TWILIO_AUTH_TOKEN=<twilio-auth-token>
TWILIO_WHATSAPP_FROM=whatsapp:+14155238886
TWILIO_ADMIN_WHATSAPP_TO=whatsapp:+91<your-10-digit-mobile>
TWILIO_ADMIN_SMS_TO=+91<your-10-digit-mobile>

# ── WEB PUSH VAPID ────────────────────────────────────────────
NEXT_PUBLIC_VAPID_PUBLIC_KEY=<vapid-public-key>
VAPID_PRIVATE_KEY=<vapid-private-key>
VAPID_SUBJECT=mailto:admin@savikaspice.in

# ── APP CONFIG ────────────────────────────────────────────────
NEXT_PUBLIC_APP_URL=https://savikaspice.vercel.app
NEXT_PUBLIC_APP_NAME=Savika Premium Spices
ADMIN_EMAIL=admin@savikaspice.in
```

SERVICE 1 Supabase *[supabase.com]*

INFO: Supabase provides database, auth, storage, and realtime. It replaces Firebase, Prisma, and NextAuth combined. Free tier covers everything Savika needs at launch.

Create Project

12. Go to supabase.com, sign up with GitHub
13. Click New Project. Name: savika-spices. Region: Southeast Asia (Singapore). Generate and save database password.
14. Wait 2 minutes for project to provision

Get API Keys

15. Go to Project Settings > API
16. **NEXT_PUBLIC_SUPABASE_URL:** Copy Project URL
17. **NEXT_PUBLIC_SUPABASE_ANON_KEY:** Copy anon public key
18. **SUPABASE_SERVICE_ROLE_KEY:** Copy service_role secret key. Never use on frontend.

Run Database Schema

19. Go to SQL Editor in left sidebar, click New Query
20. Paste the entire SQL block from Section 2.3 of this document
21. Click Run. Confirm all tables created with no errors.

Enable Google OAuth

22. Go to Authentication > Providers > Google, toggle ON
23. Copy the Callback URL shown (format: https://xxxx.supabase.co/auth/v1/callback). You will need this in the Google section below.
24. Leave Client ID and Secret empty for now. Return here after completing Service 2.

Create Storage Buckets

25. Go to Storage > New Bucket. Name: product-images. Toggle Public bucket ON. Click Create.
26. Go to Storage > New Bucket. Name: site-assets. Toggle Public bucket ON. Click Create.

Install Supabase in Next.js

```
npm install @supabase/supabase-js @supabase/ssr
```

Create Supabase Client Files

INFO: AI IDE: Create all three files below exactly as shown. These are the only correct patterns for Next.js App Router with Supabase SSR.

```
// lib/supabase/client.ts
import { createBrowserClient } from '@supabase/ssr'
export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  )
}
```

```
// lib/supabase/server.ts
import { createServerClient } from '@supabase/ssr'
import { cookies } from 'next/headers'
export async function createClient() {
  const cookieStore = await cookies()
  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    { cookies: {
      getAll() { return cookieStore.getAll() },
      setAll(c) { c.forEach(({ name, value, options }) =>
        cookieStore.set(name, value, options)) }
    }}
  )
}

// lib/supabase/admin.ts  <-- SERVER ONLY, never import in client components
import { createClient } from '@supabase/supabase-js'
export const supabaseAdmin = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY!
)

// middleware.ts  <-- project root
import { createServerClient } from '@supabase/ssr'
import { NextResponse, type NextRequest } from 'next/server'
export async function middleware(request: NextRequest) {
  let response = NextResponse.next({ request })
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    { cookies: {
      getAll() { return request.cookies.getAll() },
      setAll(c) { c.forEach(({ name, value, options }) => {
        request.cookies.set(name, value)
        response.cookies.set(name, value, options) }) }
    }}
  )
  await supabase.auth.getUser()
  return response
}
export const config = {
  matcher: ['/((?!_next/static|_next/image|favicon.ico).*)'],
}
```

SERVICE 2 Google OAuth *[console.cloud.google.com]*

INFO: Enables Sign in with Google for users. Free. Takes 5 minutes.

27. Go to console.cloud.google.com, sign in
28. Click project dropdown top-left > New Project. Name: savika-spices. Click Create.
29. Go to APIs and Services > Library. Search Google+ API. Click Enable.
30. Go to APIs and Services > Credentials > Configure Consent Screen. Choose External. App name: Savika. Fill support email. Save through all steps.
31. Back in Credentials, click + Create Credentials > OAuth client ID
32. **Application type:** Web application. Name: Savika Web.
33. **Authorised JavaScript origins:** Add https://savikaspice.vercel.app and http://localhost:3000
34. **Authorised redirect URIs:** Add the Supabase callback URL from Service 1. Also add https://savikaspice.vercel.app/auth/callback and http://localhost:3000/auth/callback
35. Click Create.
36. **GOOGLE_CLIENT_ID:** Copy Client ID
37. **GOOGLE_CLIENT_SECRET:** Copy Client Secret
38. Return to Supabase > Authentication > Providers > Google. Paste Client ID and Client Secret. Click Save.

SERVICE 3 Razorpay *[dashboard.razorpay.com]*

INFO: India's payment gateway. Sandbox mode is fully free for testing. Live mode requires KYC (1-2 business days).

39. Go to dashboard.razorpay.com, sign up with business email
40. Verify email. You are now in TEST mode by default.
41. Go to Settings > API Keys > Generate Test Key
42. **NEXT_PUBLIC_RAZORPAY_KEY_ID:** Copy Key ID (safe for frontend)
43. **RAZORPAY_KEY_SECRET:** Copy Key Secret (server only)
44. Go to Settings > Webhooks > Add New Webhook
45. **Webhook URL:** https://savikaspice.vercel.app/api/orders/webhook
46. **RAZORPAY_WEBHOOK_SECRET:** Create any strong random string and paste here. Save this value.
47. Check events: payment.captured, payment.failed, order.paid. Click Save.

```
npm install razorpay

// lib/razorpay.ts
import Razorpay from 'razorpay'
export const razorpay = new Razorpay({
  key_id: process.env.NEXT_PUBLIC_RAZORPAY_KEY_ID!,
  key_secret: process.env.RAZORPAY_KEY_SECRET!,
})
```

SERVICE 4 Resend Email *[resend.com]*

INFO: 3,000 free emails per month. Official Next.js SDK. 5-minute setup. Chosen over SendGrid for solo developer simplicity.

48. Go to resend.com, sign up
49. Go to Domains > Add Domain. Enter savikaspice.in.
50. Add the 3 DNS records (SPF, DKIM, DMARC) shown to your domain registrar DNS settings.
51. Click Verify Domain in Resend. Wait up to 30 minutes for propagation.

TIP: No custom domain yet? Set RESEND_FROM_EMAIL=onboarding@resend.dev temporarily and skip domain setup.

52. Go to API Keys > Create API Key. Name: savika-production. Permission: Full access.
53. **RESEND_API_KEY:** Copy the key immediately. It is shown only once.

```
npm install resend

// lib/resend.ts
import { Resend } from 'resend'
export const resend = new Resend(process.env.RESEND_API_KEY)
```

SERVICE 5 Twilio WhatsApp + SMS *[console.twilio.com]*

INFO: Sends WhatsApp and SMS to admin on every new order. Free sandbox for testing. Production requires WhatsApp Business approval (2-5 days).

54. Go to console.twilio.com, sign up. Verify your mobile number.
55. On Console home page:
56. **TWILIO_ACCOUNT_SID:** Copy Account SID
57. **TWILIO_AUTH_TOKEN:** Click eye icon to reveal. Copy Auth Token.
58. Go to Messaging > Try it out > Send a WhatsApp message
59. Send the join code from your own WhatsApp to the sandbox number shown (+14155238886)
60. **TWILIO_WHATSAPP_FROM:** whatsapp:+14155238886 (sandbox number)
61. **TWILIO_ADMIN_WHATSAPP_TO:** whatsapp:+91XXXXXXXXXX (your verified mobile number)
62. **TWILIO_ADMIN_SMS_TO:** +91XXXXXXXXXX (your mobile number for SMS)

```
npm install twilio

// lib/twilio.ts
import twilio from 'twilio'
export const twilioClient = twilio(
  process.env.TWILIO_ACCOUNT_SID,
  process.env.TWILIO_AUTH_TOKEN
)
```

SERVICE 6 Web Push VAPID Keys *[No account needed]*

INFO: Browser push notifications are free and self-contained. No third party account needed. Run once, save both keys permanently.

63. In your Next.js project terminal, run:

```
npx web-push generate-vapid-keys
# Output example:
# Public Key:  BEl62iUYgUivxIkv69yViEuiBIa-...
# Private Key: Aj9S2Q...
```

64. **NEXT_PUBLIC_VAPID_PUBLIC_KEY:** Copy Public Key exactly
65. **VAPID_PRIVATE_KEY:** Copy Private Key exactly
66. **VAPID_SUBJECT:** Set to mailto:admin@savikaspice.in

```
npm install web-push
npm install --save-dev @types/web-push

// lib/webpush.ts
import webpush from 'web-push'
webpush.setVapidDetails(
  process.env.VAPID_SUBJECT!,
  process.env.NEXT_PUBLIC_VAPID_PUBLIC_KEY!,
  process.env.VAPID_PRIVATE_KEY!
)
export { webpush }
```

4.2 Complete Notification Handler

INFO: AI IDE: Create this file at /app/api/notify/order/route.ts. This single function fires all three admin notification channels simultaneously when called after payment confirmation.

```
// app/api/notify/order/route.ts
import { resend } from '@/lib/resend'
import { twilioClient } from '@/lib/twilio'
import { webpush } from '@/lib/webpush'
import { supabaseAdmin } from '@/lib/supabase/admin'
import { NextResponse } from 'next/server'

export async function POST(req: Request) {
  const { orderId, total, userName, userEmail, items } = await req.json()
  const msg = `New Savika Order #${orderId.slice(0,8)} | Rs.${total} | ${userName}`

  await Promise.allSettled([
    // 1. Email
    resend.emails.send({
      from: process.env.RESEND_FROM_EMAIL!,
      to: process.env.ADMIN_EMAIL!,
      subject: `New Order - Rs.${total} from ${userName}`,
      html: `<h2>${msg}</h2><p>Items: ${JSON.stringify(items)}</p>`,
    }),

    // 2. WhatsApp
    twilioClient.messages.create({
```

```
      from: process.env.TWILIO_WHATSAPP_FROM!,
      to: process.env.TWILIO_ADMIN_WHATSAPP_TO!,
      body: msg,
    }),

    // 3. Browser push
    supabaseAdmin.from('push_subscriptions').select('subscription').then(({ data }) =>
      Promise.allSettled(
        (data || []).map(({ subscription }) =>
          webpush.sendNotification(subscription,
            JSON.stringify({ title: 'New Savika Order!', body: msg }))
        )
      )
    ),
  ])

  return NextResponse.json({ ok: true })
}
```

4.3 CSV Export Routes

```
// app/api/export/customers/route.ts
import { supabaseAdmin } from '@/lib/supabase/admin'
import { NextResponse } from 'next/server'

export async function GET() {
  const { data } = await supabaseAdmin
    .from('profiles')
    .select('full_name, email, mobile, address, created_at')
    .order('created_at', { ascending: false })

  const csv = [
    ['Name', 'Email', 'Mobile', 'Address', 'Joined'].join(','),
    ...(data || []).map(r => [
      r.full_name ?? '', r.email ?? '', r.mobile ?? '',
      JSON.stringify(r.address ?? {}), r.created_at
    ].map(v => `"${String(v).replace(/"/g, '""')}"`).join(','))
  ].join('\n')

  return new NextResponse(csv, {
    headers: {
      'Content-Type': 'text/csv',
      'Content-Disposition': 'attachment; filename=savika-customers.csv'
    }
  })
}
```

4.4 Vercel Environment Variables

CRITICAL: All env variables must be added to Vercel separately from .env.local. Vercel does not read local files. Without this step, production will fail.

67. Go to vercel.com/dashboard and open the Savika project
68. Click Settings > Environment Variables
69. Click Import .env file and upload your completed .env.local directly
70. Set Environment to Production + Preview + Development for all variables
71. Click Save
72. Go to Deployments tab, click three dots on latest deployment, click Redeploy

4.5 Variable Security Reference

Variable	Frontend Safe?	Where to Use
NEXT_PUBLIC_SUPABASE_URL	YES	Anywhere
NEXT_PUBLIC_SUPABASE_ANON_KEY	YES	Anywhere
SUPABASE_SERVICE_ROLE_KEY	NO - NEVER	API routes and lib/supabase/admin.ts only
GOOGLE_CLIENT_ID	NO	API routes only
GOOGLE_CLIENT_SECRET	NO - NEVER	API routes only
NEXT_PUBLIC_RAZORPAY_KEY_ID	YES	Anywhere (needed for checkout modal)
RAZORPAY_KEY_SECRET	NO - NEVER	API routes only
RAZORPAY_WEBHOOK_SECRET	NO - NEVER	Webhook route only
RESEND_API_KEY	NO - NEVER	API routes only
TWILIO_ACCOUNT_SID	NO - NEVER	API routes only
TWILIO_AUTH_TOKEN	NO - NEVER	API routes only
NEXT_PUBLIC_VAPID_PUBLIC_KEY	YES	Anywhere (needed for push registration)
VAPID_PRIVATE_KEY	NO - NEVER	API routes only

PART 5

Go-Live Checklist

Complete every item before launching Savika

CRITICAL: Tick every box before announcing the site publicly. Each item is ordered by dependency.

#	Task	Section	Done?
1	.env.local created in project root with all 20 variables	4.1	[ ]
2	.env.local confirmed inside .gitignore	4.1	[ ]
3	Supabase project created (Singapore region)	Service 1	[ ]
4	All SQL from Section 2.3 executed with no errors	2.3	[ ]
5	Supabase Storage buckets created (product-images, site-assets)	Service 1	[ ]
6	Google OAuth credentials created and pasted into Supabase	Service 2	[ ]
7	Google OAuth redirect URIs include both localhost and production URL	Service 2	[ ]
8	Razorpay test keys generated	Service 3	[ ]
9	Razorpay webhook URL configured and test event sent	Service 3	[ ]
10	Resend account created, domain verified or fallback email set	Service 4	[ ]
11	Twilio account created, WhatsApp sandbox joined on your phone	Service 5	[ ]
12	VAPID keys generated and both copied to .env.local	Service 6	[ ]
13	All 20 env variables added to Vercel dashboard	4.4	[ ]
14	Vercel redeployed after adding all variables	4.4	[ ]
15	Poppins font applied globally, no other fonts in codebase	3.1	[ ]
16	All em dashes removed from entire codebase	1.2	[ ]

#	Task	Section	Done?
17	All images use next/image component	2.1	[ ]
18	Skeleton screens on product listing, homepage banners, order list	3.5.2	[ ]
19	Test user signup (email) and confirm profile row in Supabase	2.5	[ ]
20	Test Google OAuth login end to end	2.5	[ ]
21	Test checkout with Razorpay test card (4111 1111 1111 1111)	2.6	[ ]
22	Confirm all 3 admin notifications received on test order	1.5.5	[ ]
23	CSV export tested from admin Customers page	1.5.3	[ ]
24	Admin Updates page: upload banner image, change text, confirm on homepage	1.5.4	[ ]
25	Lighthouse score checked at PageSpeed Insights >= 90 all metrics	1.5.6	[ ]
26	Razorpay switched to LIVE keys after KYC approval	Service 3	[ ]
27	Twilio WhatsApp upgraded from sandbox to business number	Service 5	[ ]
28	sitemap.xml and robots.txt present and accessible	1.5.6	[ ]

Savika is ready to launch.

Every requirement defined. Every service documented. Every key mapped. Every checklist item verified.